

Checking the $k = 9$, $\ell = 6$ tables for the search of exp

What `code/exptable9` checks on each line, in the notation of `doc/htr.md`

Paul Zimmermann's search for the hard-to-round cases of `exp` (`doc/htr.md`) reads tables with one line per subrange of inputs. This note is about his tables for every binary64 input, with Taylor polynomials of $k = 9$ terms and coefficients of $\ell = 6$ words. It says what the tables hold, the six conditions under which the search misses no hard case on a line, and how the Rocq files in `code/exptable9/` check them with one boolean function whose answer `true` is proved to imply the six conditions. The same method was used on the smaller table `in_lt` (`doc/exptable-note.typ`); this note is complete by itself.

1 Summary

- **The sample passes.** 22 files of the archive, 7 136 lines (about 1% of it), chosen over the whole range: the largest binades, x near ± 1 , negative x , and subnormal outputs. For each file, Rocq proves that every line, read as its text says, satisfies the six conditions (section 5) under which the search misses no hard case on it. No line fails. (This run used a window larger by 2^{320} ; with the E below it is to be made again, section 11.)
- **E is inferred, not read from the search program** (section 6): $E = 0x600000 \cdot 2^{320}$, exactly $3 \cdot 2^{-43}$ in units of β^ℓ . The n of each line keeps both the rounding term and the Taylor term at or below 2^{-43} , so the window must hold three terms of 2^{-43} . With the window of the program for `in_lt`, `0x400001`, the lines near $x = \pm 1$ fail the window condition, although their B_i are right. The E of the program that reads these tables is still to be confirmed.
- **The bit-flip test passes** (section 11): changing one bit of any coefficient B_i , or moving x_0 , makes the check fail.
- **The whole archive** (687 184 lines) would take about 19.5 hours of processor time, about 1.3 hours on 15 workers of `roquableu` (scaled from the sample, not measured).

2 Proof by a checker, for a reader who does not use Rocq

Rocq is a proof assistant: it accepts a theorem only with a proof it can verify step by step. It is also a programming language, and it can run its own programs. The files here use both sides:

- `check_raw` is a **program**. It reads lines of a table, as text, and answers `true` or `false`. It uses integers only, plus one function of the Interval library that returns two numbers $L \leq \exp(x) \leq U$.
- `check_rawP` is a **theorem** about that program: **if `check_raw` answers `true`, then every line satisfies the six conditions**. It is proved once, for every possible table.
- To prove that a table file satisfies the six conditions, Rocq runs the program on it, sees `true`, and applies the theorem.

So one does not need to trust the program, or to read it: what one must read is the statement of the six conditions (section 5). The run is done by the evaluator `native_compute`, which compiles the program to machine code.

Two words used below. The **binade** of a positive number y is the interval $[2^e, 2^{e+1})$ that contains it. An **enclosure** of y is a pair of numbers $L \leq y \leq U$, here with $U - L$ tiny compared with y .

3 The tables

The archive `all.tar.bz2` holds 2 203 files and 687 184 lines. The file `ine` is for the inputs x with $\exp(x)$ in $[2^{e-1}, 2^e)$: from `in1024`, where $\exp(x)$ reaches the largest double, down to `in-1074`, where $\exp(x)$ is below the smallest positive subnormal double; around $x = 0$, where the subranges are short,

the binades are split into several files (`in0-0`, `in0-1`, ...). Most files have 294 lines, those near $x = 0$ have 257 to 665.

A line is x_0 (a double, in hexadecimal), then n , then $k = 9$ integers $B_0, \dots, B_8$ of $\ell = 6$ words of 64 bits each, the least significant word first: 56 fields. For instance, the first line of `in2` starts

`0x1.62e42fefa39fp-1 6782714940072 0xb2a413074401254d 0x6ac68b078499736e ...`

Over the 687 184 lines (all measured with Python on the files):

- x_0 is always a normal double, positive or negative (351 115 lines are negative), with $|x_0|$ from 2^{-53} to 709.8;
- a few lines have no fraction digits, like `-0x1p-17`, and some have $n = 0$;
- no subrange $[x_0, x_1]$ leaves the binade of x_0 .

4 The notation, and what changes from `in_lt`

- $\beta = 2^{64}$ is the word basis, ℓ the number of words of a coefficient.
- A subrange is $[x_0, x_1]$ with $x_1 = x_0 + nu$, where $u = \text{ulp}(x)$ is the same for all x in it. The search looks at the $n + 1$ inputs $x_0 + ju$, $j = 0, \dots, n$.
- $v = \frac{1}{2} \text{ulp}(\exp x)$, the same for all x in $[x_0, x_1]$.
- $a_i = \exp(x_0)u^i/i!$, $0 \leq i < k$, the Taylor coefficients.
- A_i is an integer with $0 \leq A_i < \beta^\ell$ and $|A_i - \beta^\ell \text{frac}(a_i/v)| < 1$, and $P(j) = A_0 + A_1j + \dots + A_{k-1}j^{k-1}$.
- A line holds $B_i = P(i) \bmod \beta^\ell$ for $i < k$: the values of P at $0, \dots, k - 1$, not the A_i (step 3 of the search of `doc/htr.md`).
- $m = 43$: an input is hard to round when $\exp(x)/v$ is within 2^{-m} of an integer. E is the window of the search.

Three things are more general than for `in_lt`, where every x_0 was positive and in $[2^9, 2^{10}]$:

- **The sign of x , and u , vary.** A line gives $x_0 = S_0 \cdot 2^{e_x - 52}$ with S_0 an integer, $2^{52} \leq |S_0| < 2^{53}$, negative when x_0 is; then $u = 2^{e_x - 52}$ and the inputs are $x_0 + ju = (S_0 + j)2^{e_x - 52}$. Both S_0 and e_x are read from the line. The search always goes upwards: $u > 0$ and $x_1 \geq x_0$, also when x_0 is negative, where it goes towards 0 ($|S_0 + j|$ decreases). (The other choice would have been to search away from 0, with $x_1 < x_0$ for a negative x_0 .) Condition 1 checks it on every line: S_0 and $S_0 + n$ have the same sign and both have 53 bits.
- **v for small outputs.** If $2^e \leq \exp(x) < 2^{e+1}$ with $e \geq -1022$, the doubles near $\exp(x)$ are the multiples of 2^{e-52} , and $v = 2^{e-53}$. Below 2^{-1021} , the subnormal doubles and those of the smallest normal binade are all multiples of 2^{-1074} : there $v = 2^{-1075}$ whatever e . Both cases are

$$v = 2^{\max(e, -1022) - 53}.$$

- $k = 9$, $\ell = 6$, and E (section 6).

5 The six conditions

For one line, the search misses no hard case when:

1. **u is constant:** $0 \leq n$, and S_0 and $S_0 + n$ have the same sign and absolute values in $[2^{52}, 2^{53})$, so every $x_0 + ju$ has exponent e_x ;
2. **v is constant:** $2^e \leq \exp(x_0)$ and $\exp(x_1) < 2^{\max(e, -1022) + 1}$, so $v = 2^{\max(e, -1022) - 53}$ fits every $\exp(x)$ of the subrange;
3. **(H_A):** there are $A_0, \dots, A_{k-1}$ with $0 \leq A_i < \beta^\ell$ and $|A_i - \beta^\ell \text{frac}(a_i/v)| < 1$, and the line holds $B_i = P(i) \bmod \beta^\ell$ for $i < k$;
4. **(H_T):** for $0 \leq j \leq n$,

$$|\exp(x_0 + ju)/v - (a_0 + a_1j + \dots + a_{k-1}j^{k-1})/v| \leq \rho, \quad \rho = \exp(x_1) \frac{(nu)^k}{k!v};$$

5. **(H_E):** $E \geq \beta^\ell(2^{-m} + \rho) + (1 + n + \dots + n^{k-1})$;
6. **the window is less than half the modulus:** $2E < \beta^\ell$.

Conditions 3 to 6 are the hypotheses of `hscan_exp` in `code/APaul/rocq/TaylorLink.v`, which says the search then misses no $x_0 + ju$ with $\exp(x_0 + ju)/v$ within 2^{-m} of an integer. In Rocq, the six conditions are `line_ok S0 ex n B in ExpCheck.v` (annex A).

6 The parameters, and the value of E

They are the first definitions of `ExpCheck.v`:

```

Definition beta : Z := 2 ^ 64.      (* the machine word basis          *)
Definition l : Z := 6.              (* words of an A_i or a B_i      *)
Definition k : nat := 9.            (* terms of the Taylor polynomial *)
Definition m : Z := 43.            (* identical bits after the round bit *)
Definition E : Z := 0x600000 * 2 ^ 320. (* the window (inferred) *)
Definition guard : Z := 64.        (* extra bits in the exp enclosure *)

```

E is not in the tables, and the program that reads these tables is not in the repository: **E is inferred from the tables**. Over the 687 184 lines, the two error terms of condition 5 each reach 2^{-43} exactly and never more (computed with Python from n and e_x alone, bounding $\exp(x_1)/v < 2^{54}$):

term	largest value	line
$(1 + n + \dots + n^8)\beta^{-\ell}$	$2^{-43.0000}$	in-1, $x_0 = -0x1.62e42fefa39efp+0$
$\rho \leq 2^{54}(nu)^9/9!$	$2^{-43.0000}$	in-2, $x_0 = -0x1.0a2b23f3bab73p+1$

So the n of each line is chosen to keep both terms below 2^{-43} , and the window must hold three terms of 2^{-43} : $E \geq \beta^\ell \cdot 3 \cdot 2^{-43}$. The search program `htr3.c` for `in_lt` writes its window as a constant added to the top word, rounded up: `ERR = 0x400001` for $\lceil 2^{64}(2^{-43} + 2^{-43} + 2^{-90}) \rceil$. The same rule here gives $\lceil 2^{64} \cdot 3 \cdot 2^{-43} \rceil = 0x600000$: the bound is an integer, there is nothing to round, and $E = 0x600000 \cdot 2^{320} = \beta^\ell \cdot 3 \cdot 2^{-43}$ (about $2^{342.585}$). Condition 5 is checked on every line with this value; a wrong guess would make the check fail, never pass. The value of `htr3.c`, `0x400001`, is too small here: the lines near $x = \pm 1$ fail condition 5 with it (while their B_i are right).

7 How one line is checked: `check_line`

`check_raw` reads a line with `parse` into (S_0, e_x, n, B) , then runs `check_line`, with integers only, except for two calls to the certified `exp` of the Interval library.

1. **Condition 1:** integer tests on n , $|S_0|$, $|S_0 + n|$ and the sign of $S_0(S_0 + n)$.
2. **Enclosures.** `encl` builds the double $S_0 2^{e_x-52}$ as a float of Interval and calls its `exp` at $384 + 53 + 64 = 501$ bits: $L_0 \leq \exp(x_0) \leq U_0$, and the same at x_1 .
3. **Condition 2.** e is the exponent of L_0 ; the test is $U_1 < 2^{\max(e, -1022)+1}$.
4. **Condition 3.** For each i , $T = \beta^\ell a_i / v$ is $\exp(x_0)$ times a power of two, divided by $i!$. At both ends of the enclosure the checker computes the nearest integer R of T and the integer part q of T/β^ℓ , and requires them equal; then $A_i = R - q\beta^\ell$ is within $1/2$ of $\beta^\ell \text{frac}(a_i/v)$. It computes $P(0), \dots, P(8)$ modulo β^ℓ and compares them with the B_i .
5. **Condition 4** needs no test: it is the Taylor theorem (`ExpTaylor.v`).
6. **Condition 5** is one integer inequality, with $\exp(x_1)$ replaced by U_1 .
7. **Condition 6** is checked once by `check_raw`.

8 Three lines

The first line of three files: the largest inputs, inputs near 1, and subnormal outputs. The rows S_0 to the enclosure, and the last one, are the checker's own values; x_0 in decimal and the window terms are computed with Python.

	in1024	in2	in-1074
x_0	709.089566	0.693147 (ln 2)	-745.133219
S_0	6237217781087073	6243314768165360	-6554261109157969
e_x, u	9, 2^{-43}	-1, 2^{-53}	9, 2^{-43}
n	20778149366	6782714940072	20778149366
e	1023	1	-1075
v	2^{970}	2^{-52}	2^{-1075}
enclosure of $\exp(x_0)$	502 bits, width 2	502 bits, width 2	520 bits, width 271017
ρ	$2^{-44.00}$	$2^{-58.84}$	$2^{-97.00}$
$1 + \dots + n^8$	$2^{274.20}$	$2^{341.00}$	$2^{274.20}$
window needed	$2^{341.59}$	$2^{342.00}$	$2^{341.00}$
check_raw	true	true	true

The enclosure is $L_0 = p \cdot 2^f$ with p of the number of bits shown, and $U_0 = L_0 + w \cdot 2^f$ with w the width shown. The target term $\beta^\ell 2^{-43} = 2^{341}$ is in every window. At **in1024** the Taylor term ρ is near its bound 2^{-43} ; at **in2** the rounding of the A_i is, since n is large; at **in-1074**, $\exp(x_0)$ is in $[2^{-1075}, 2^{-1074})$, so $\exp(x)/v$ stays between 1 and 2, and ρ is tiny. All three need less than $E \approx 2^{342.585}$.

9 Line 1 of in-1074, step by step

The first line of **in-1074** is (the B_i are shown by their most significant word only; each has six):

$$x_0 = -0x1.74910d52d3051p+9 \approx -745.133219, \quad n = 20778149366,$$

$B_0 = 0x1bec60 \dots$	$B_1 = 0x3bec60 \dots$	$B_2 = 0x5bec60 \dots$
$B_3 = 0x7bec60 \dots$	$B_4 = 0x9bec60 \dots$	$B_5 = 0xbbec60 \dots$
$B_6 = 0xdbec60 \dots$	$B_7 = 0xfbec60 \dots$	$B_8 = 0x11bec60 \dots$

It is the case **in_lt** does not have: x is negative and $\exp(x)$ is below the smallest positive double. The run of section 11 checks this line as it checks all the others: **check_raw** reads the text with **parse**, then gives the numbers to **check_line**. The numbers below are the checker's own values when it says so; the others are computed with **check_line.py**'s integer exp at 1200 bits (Python, not a proof).

Reading the line. **parse** finds the sign -, the fraction digits 74910d52d3051, the exponent +9, then n in decimal and 54 words $0x\dots$, grouped by six, the least significant first, into $B_0, \dots, B_8$ (section 10 states and proves this reading). So $S_0 = -(16^{13} + 0x74910d52d3051) = -6554261109157969$ and $e_x = 9$.

Condition 1: u is constant. $u = 2^{9-52} = 2^{-43}$. The inputs are $(S_0 + j)2^{-43}$ for $j = 0, \dots, n$, from $x_0 \approx -745.133219$ to $x_1 = (S_0 + n)2^{-43} \approx -745.130857$, with $S_0 + n = -6554240331008603$. Both significands are negative with absolute values in $[2^{52}, 2^{53})$, so every input has exponent 9: the search walks towards 0 without leaving the binade $(-2^{10}, -2^9]$.

Enclosures of $\exp$ (checker's values). Interval's exp at 501 bits (β^ℓ , 384 bits, plus 53, plus guard = 64) returns, for x_0 , $L_0 = p_0 \cdot 2^{-1594}$ with p_0 an integer of 520 bits and $U_0 = L_0 + 271017 \cdot 2^{-1594}$, a relative width of about 2^{-501} ; at x_1 , $L_1 = p_1 \cdot 2^{-1594}$, p_1 of 520 bits, and $U_1 = L_1 + 266797 \cdot 2^{-1594}$.

Condition 2: v is constant. L_0 is in $[2^{-1075}, 2^{-1074})$, so $e = -1075$: $\exp(x_0)$ is below the smallest normal double 2^{-1022} , and $v = 2^{\max(-1075, -1022)-53} = 2^{-1075}$, half the distance 2^{-1074} between two subnormal doubles. The test is $U_1 < 2^{-1022+1}$; in fact $U_1 < 2^{-1074}$, so every $\exp(x)$ of the line stays in $[2^{-1075}, 2^{-1074})$: between 0 and the smallest positive double, $\exp(x)/v$ goes from 1 to $\exp(x_1)/v \approx 1.002365$.

Condition 3: the A_i , then the B_i . For each i the checker forms $T = \beta^\ell a_i/v$ at both ends of the enclosure, finds the same integer part q of T/β^ℓ and the same nearest integer R of T , and takes $A_i = R - q\beta^\ell$:

i	a_i/v	q	A_i , top word
0	$1 + 2^{-43.20}$	1	0x1bec60
1	$1.14 \cdot 10^{-13} \approx 2^{-43}$	0	0x200000
2	$6.46 \cdot 10^{-27} \approx 2^{-87}$	0	0x0
3	$2.45 \cdot 10^{-40}$	0	0x0
4	$6.96 \cdot 10^{-54}$	0	0x0
5	$1.58 \cdot 10^{-67}$	0	0x0
6	$3.00 \cdot 10^{-81}$	0	0x0
7	$4.87 \cdot 10^{-95}$	0	0x0
8	$6.92 \cdot 10^{-109}$	0	0x0

The top words of the A_i are the checker's; the other columns are Python's. Here $a_i/v = (a_0/v)2^{-43i}/i!$, and only A_0 and A_1 reach the top word. The checker then computes $P(0), \dots, P(8)$ modulo β^ℓ and finds exactly the nine B_i of the line (checker's value `true`): on the top word, $P(i) = A_0 + iA_1 + \dots$ is `0x1bec60 + i · 0x200000`, which is the column of top words above.

Note $a_0/v = \exp(x_0)/v = 1 + 2^{-43.20}$: $\exp(x_0)$ is within 2^{-43} of $v = 2^{-1075}$, the midpoint between 0 and the smallest positive double. So x_0 itself is a candidate of the search ($j = 0$). This is expected: the file starts at the first double whose `exp` is at least 2^{-1075} , and one step u of x moves $\exp(x)$ by a relative 2^{-43} .

Condition 4: (H_T). The Taylor theorem gives the bound, with

$$\rho = \exp(x_1) \frac{(nu)^9}{9!v} \approx 2^{-97.00},$$

tiny, because $\exp(x_1)/v$ is only about 1 here, where it is about 2^{53} for a normal output.

Condition 5: (H_E). The three terms of the window are $\beta^\ell 2^{-43} = 2^{341}$, $\beta^\ell \rho \approx 2^{287}$ and $1 + n + \dots + n^8 \approx 2^{274.20}$: about $2^{341.0000}$ in all. The checker replaces $\exp(x_1)$ by $U_1 = q_1 \cdot 2^{-1594}$, with $q_1 = p_1 + 266797$, and tests one integer inequality (`window_ok`). With $\beta^\ell \rho \leq q_1 n^9 2^g / 9!$ where $g = -1594 + 384 - 43 \cdot 9 - (-1075) = -522$ (checker's value), and $C = 2^{341} + (1 + n + \dots + n^8)$, it tests

$$q_1 n^9 + C \cdot 9! \cdot 2^{522} \leq E \cdot 9! \cdot 2^{522},$$

the window condition multiplied by $9! \cdot 2^{522}$, with $E = 0x600000 \cdot 2^{320}$ (checker's value `true`).

Condition 6 is on the whole table: $2E = 0xc00000 \cdot 2^{320} < \beta^\ell = 2^{384}$.

On this line `check_line` answers `true`; the line then satisfies the six conditions, by the theorem.

10 The theorems

```
Theorem check_rawP ls : check_raw ls = true ->
  (2 * E < beta_l)%Z /\
  forall s, In s ls -> exists S0 ex n B,
    parse s = Some (S0, ex, n, B) /\ line_ok S0 ex n B.
```

```
Theorem parse_sound s S0 ex n B :
  parse s = Some (S0, ex, n, B) -> line_text (to_list s) S0 ex n B.
```

`line_text` (`ExpParseSpec.v`, annex D) says in plain definitions what the text of a line means: an optional `-`, `0x1`, an optional `.` followed by 0 to 13 hexadecimal digits F of value f , p , a signed decimal

exponent e_x , n in decimal, then the 54 words $0x\dots$; and $S_0 = \pm(16^{13} + f \cdot 16^{13-|F|})$, $B_i = w_{6i} + w_{6i+1}\beta + \dots + w_{6i+5}\beta^5$.

For each table file `ine` given to `mksample.sh`, the generated file `SampleAll.v` proves

```
Lemma lines_ok_N : forall s, In s Data_N.lines ->
  exists S0 ex n B, line_text (to_list s) S0 ex n B /\ line_ok S0 ex n B.
```

(N is the file name, with `in-1074` written `inm1074` and `in0-0` written `in0_0`): **every line of the file, read as its text says, satisfies the six conditions.** No lemma is admitted; the theorems rest on the axioms of Rocq’s real numbers, functional extensionality, the excluded middle, and the primitive integers and strings.

11 The sample, and running it

`sample.tar.xz` holds 22 files of the archive, 7 136 lines, about 1% of it, chosen over the whole range: `in1024`, `in1023`, `in1000`, `in900`, `in700`, `in500`, `in300`, `in100`, `in10` (large inputs), `in2`, `in1-0`, `in0-0`, `in-1` (near $x = 0$), `in-100`, `in-500`, `in-900`, `in-1000` (negative inputs), `in-1021`, `in-1022`, `in-1023`, `in-1050`, `in-1074` (around and below the smallest normal output).

```
tar xJf sample.tar.xz
./mksample.sh in1024 in1023 ... in-1074
make -j15 sample          # one Run_N.v per file, native_compute
make sample-all          # SampleAll.v
```

`Run_N.v` depends on `ExpCheck.v` and `ExpParse.v` only, so the proofs can change without running the tables again.

The sample was run on `roquableu` (an Intel Xeon E5-2667 server, 24 threads) with `make -j15 sample`, for $E = 0x600001 \cdot 2^{320}$, a window larger by 2^{320} ; all 22 files pass. With $E = 0x600000 \cdot 2^{320}$ the run is to be made again; on the desktop, the first line of `in1024`, `in2`, `in-1074`, a line of `in0-*` with no fraction digits, and the two lines where the rounding term and the Taylor bound are largest (section 6) pass. Measured:

Qed of a file of 294 lines	26 to 34 s
Qed of <code>in2</code> , <code>in-1</code> (665 lines)	57 s, 65 s
<code>make -j15 sample</code> , wall	1 min 11 s
<code>make -j15 sample</code> , processor	12 min 10 s
<code>make sample-all</code> (with <code>SampleAll.v</code>)	12.6 s

`make sample-all` then builds `SampleAll.v`: for each of the 22 files, every line, read as its text says, satisfies the six conditions.

That is about 0.1 s of processor time a line. At that rate the whole archive, 687 184 lines, would take about 19.5 hours of processor time, about 1.3 hours on 15 workers (scaled, not measured).

The bit-flip test (`ExpMutate.v`, `make test`) makes sure every coefficient and x_0 is really checked. On the first line of `in1024`, `in2` and `in-1074`, which pass, it changes one bit of one B_i (the lowest bit, bit 192 and the top bit 383, for each of the 9 coefficients), or moves x_0 by one ulp either way, or changes bit 0, 20 or 51 of $|S_0|$ (for a negative S_0 , of its two’s complement): 32 changes a line. Rocq checks by evaluation that `check_line` answers **false** on each of them, in 13.5 s on the desktop (measured).

12 What is not checked

- The rest of the archive (the sample is about 1% of it).

- That the lines of a file, or of the archive, cover the whole range of inputs (the files of `in_lt` have such a check, `ExpCover.v` in `code/exptable`; here the subranges change binade of x within a file, and the check is not written).
- The value of E of Paul's program (section 6): E is inferred.
- The search itself: that is `TaylorLink.v`, which this check feeds.

A ExpCheck.v

```
(** * Checking a table for the hard-to-round search of exp, by reflection
```

The search is the one of [doc/htr.md] (Paul Zimmermann), whose notation we keep. The inputs are cut into subranges $[x_0, x_1]$ with $x_1 = x_0 + n u$, $[u = \text{ulp } x]$ and $[v = \text{ulp}(\exp x) / 2]$ constant on each. With $[a_i = \exp x_0 u^i / i!]$, each $[A_i]$ is an integer with $[0 \leq A_i < \beta^l]$ and $[|A_i - \beta^l \text{frac}(a_i/v)| < 1]$, and $[P(j) = A_0 + A_1 j + \dots + A_{(k-1)} j^{(k-1)}]$.

A line of the table is $[(x_0, n, B)]$ with $[B_i = P(i) \bmod \beta^l]$ for $[i < k]$. Here $[x_0]$ is any normal double, positive or negative, and $[\exp x_0]$ may be subnormal. $[x_0]$ is written $[S_0 2^{(ex - 52)}]$, $[S_0]$ an integer with $[2^{52} \leq |S_0| < 2^{53}]$; then $[u = 2^{(ex - 52)}]$ and the inputs are $[x_0 + j u = (S_0 + j) 2^{(ex - 52)}]$, $[j = 0 \dots n]$.

$[line_ok]$ states the six conditions under which the search on a line misses no hard case; $[check_line]$ tests them. This file holds the definitions only; $[ExpProof]$ proves the checker correct. *)

```
From Stdlib Require Import ZArith Reals Lia Lra List.
From Flocq Require Import Core.
From Bignums Require Import BigZ.
From Interval Require Import Xreal Basic Specific_bigint Specific_ops.
From Interval Require Import Float_full.
Import ListNotations.
```

```
(** ** Parameters
```

The table and the search are described here and nowhere else, in the notation of [doc/htr.md]. *)

```
Open Scope Z_scope.
```

```
Definition beta : Z := 2 ^ 64.      (* the machine word basis          *)
Definition l : Z := 6.              (* words of an A_i or a B_i      *)
Definition k : nat := 9.            (* terms of the Taylor polynomial *)
Definition m : Z := 43.            (* identical bits after the round bit *)
Definition E : Z := 0x600000 * 2 ^ 320. (* the window (inferred) *)
Definition guard : Z := 64.        (* extra bits in the exp enclosure *)
```

```
(** ** Derived constants and the binary64 format
```

$[\beta^l = 2^{l \text{bits}}]$; a double has $[prec]$ bits, the smallest normal exponent is $[emin]$: if $[2^e \leq y < 2^{(e+1)}]$, the doubles near $[y]$ are multiples of $[2^{(\max e \text{ emin} - prec + 1)}]$, so $[v]$ is $[2^{(\max e \text{ emin} - prec)}]$. *)

```
Module SFBI2 := SpecificFloat BigIntRadix2.
Module I := FloatIntervalFull SFBI2.
```

```
Definition beta_l : Z := beta ^ l.
Definition lbits : Z := Z.log2 beta * l.
Definition prec : Z := 53.          (* bits of a double                *)
Definition emin : Z := -1022.       (* smallest normal exponent         *)
```

```

Definition iprec : SFBI2.precision :=
  SFBI2.PtoP (Z.to_pos (lbits + prec + guard)).

(** [u = 2^uexp ex] for [x0] in [[2^ex, 2^(ex+1))]. *)
Definition uexp (ex : Z) : Z := ex - (prec - 1).

(** [v = 2^vexp e] for [exp x] in [[2^e, 2^(e+1))], or below [2^emin]. *)
Definition vexp (e : Z) : Z := Z.max e emin - prec.

(** ** The conditions **)

Open Scope R_scope.

Definition bp (e : Z) : R := bpow radix2 e.
Definition fracR (r : R) : R := r - IZR (Zfloor r).

(** [a_i = exp x0 u^i / i!]. *)
Definition a (x0 u : R) (i : nat) : R := exp x0 * u ^ i / INR (fact i).

(** [f 0 + ... + f (N-1)]. *)
Fixpoint sumR (f : nat -> R) (N : nat) : R :=
  match N with 0 => 0 | S N' => sumR f N' + f N' end.

(** [P(j) = A_0 + A_1 j + ... + A_(k-1) j^(k-1)], in integers. *)
Definition Pz (A : list Z) (j : Z) : Z :=
  fold_right Z.add 0%Z
    (map (fun i => (nth i A 0 * j ^ Z.of_nat i)%Z) (seq 0 k)).

(** The six conditions on a line [(x0, n, B)], [x0 = S0 2^(ex - 52)]. *)
Definition line_ok (S0 ex n : Z) (B : list Z) : Prop :=
  let u := bp (uexp ex) in
  let x0 := IZR S0 * u in
  let x1 := IZR (S0 + n) * u in
  exists e : Z,
  let v := bp (vexp e) in
  let rho := exp x1 * (IZR n * u) ^ k / (INR (fact k) * v) in
  (* 1. u is constant: every S0 + j, j = 0 .. n, has 53 bits *)
  (0 <= n)%Z /\ (2 ^ 52 <= Z.abs S0 < 2 ^ 53)%Z /\
  (2 ^ 52 <= Z.abs (S0 + n) < 2 ^ 53)%Z /\ (0 < S0 * (S0 + n))%Z /\
  (* 2. v is constant on [exp x0, exp x1] *)
  bp e <= exp x0 /\ exp x1 < bp (Z.max e emin + 1) /\
  (* 3. (H_A), and the table holds P(0) .. P(k-1) *)
  (exists A : list Z, length A = k /\ length B = k /\
    (forall i, (i < k)%nat ->
      (0 <= nth i A 0%Z < beta_l)%Z /\
      Rabs (IZR (nth i A 0%Z) - IZR beta_l * fracR (a x0 u i / v)) < 1) /\
    (forall i, (i < k)%nat ->
      nth i B 0%Z = (Pz A (Z.of_nat i) mod beta_l)%Z)) /\
  (* 4. (H_T), the Taylor remainder *)
  (forall j, (0 <= j <= n)%Z ->
    Rabs (exp (x0 + IZR j * u) / v -
      sumR (fun i => a x0 u i / v * IZR j ^ i) k) <= rho) /\
  (* 5. (H_E), the window *)
  IZR beta_l * (bp (- m) + rho) + sumR (fun i => IZR n ^ i) k <= IZR E.

Close Scope R_scope.

```

```

(** ** The checker *)

Open Scope bool_scope.

(** The float  $[N \ 2^{ue}]$ . *)
Definition xpt (N ue : Z) : SFBI2.type :=
  Specific_ops.Float (BigZ.of_Z N) (BigZ.of_Z ue).

(** A float as its signed significand and exponent. *)
Definition getS (f : SFBI2.type) : option (Z * Z) :=
  match SFBI2.toF f with
  | Basic.Float s p e => Some (if s then Z.neg p else Z.pos p, e)
  | _ => None
  end.

(** A positive float as its significand and exponent. *)
Definition getF (f : SFBI2.type) : option (positive * Z) :=
  match SFBI2.toF f with Basic.Float false p e => Some (p, e) | _ => None end.

(** An enclosure  $[[mL \ 2^{fL}, mU \ 2^{fU}]]$  of  $[exp \ (N \ 2^{ue})]$ . *)
Definition encl (N ue : Z) : option (Z * Z * Z * Z) :=
  if SFBI2.valid_lb (xpt N ue) && SFBI2.valid_ub (xpt N ue) &&
    match getS (xpt N ue) with
    | Some (p, e) => Z.eqb p N && Z.eqb e ue
    | None => false
    end
  then
    match I.exp iprec (Float.Ibnd (xpt N ue) (xpt N ue)) with
    | Float.Ibnd lo up =>
      if SFBI2.valid_lb lo && SFBI2.valid_ub up then
        match getF lo, getF up with
        | Some (pL, fL), Some (pU, fU) => Some (Z.pos pL, fL, Z.pos pU, fU)
        | _, _ => None
        end
      else None
    | Float.Inan => None
    end
  else None.

(**  $\lfloor p \ 2^g / fk + h / 2 \rfloor$ , in integers. *)
Definition flr (p g fk h : Z) : Z :=
  if Z.leb 0 g then (2 * p * 2 ^ g + h * fk) / (2 * fk)
  else (2 * p + h * fk * 2 ^ (- g)) / (2 * fk * 2 ^ (- g)).

(**  $[p \ 2^f < 2^g]$ , for  $[p \geq 1]$ . *)
Definition lt2 (p f g : Z) : bool :=
  if Z.leb g f then false else Z.ltb p (2 ^ (g - f)).

(**  $[A_i]$ , from the enclosure of  $[exp \ x0]$ :  $[beta^l \ a_i / v]$  is
 $[p \ 2^{(f + gi)} / i!]$ ; its nearest integer  $[R]$  and its integer part
 $[q]$  (in units of  $[beta^l]$ ) must be the same at both ends. *)
Definition coefA (mL fL mU fU ve ue : Z) (i : nat) (fk : Z) : option Z :=
  let gi := lbits + ue * Z.of_nat i - ve in
  let R := flr mL (fL + gi) fk 1 in
  let q := flr mL (fL + gi - lbits) fk 0 in

```

```

let Ai := R - q * beta_l in
if Z.eqb R (flr mU (fU + gi) fk 1) &&
   Z.eqb q (flr mU (fU + gi - lbits) fk 0) &&
   Z.leb 0 Ai && Z.ltb Ai beta_l
then Some Ai else None.

(** [A_i .. A_(i+c-1)], [fk] being [i!]. *)
Fixpoint coefs (mL fL mU fU ve ue : Z) (i : nat) (fk : Z) (c : nat)
: option (list Z) :=
match c with
| 0 => Some nil
| S c' =>
  match coefA mL fL mU fU ve ue i fk,
        coefs mL fL mU fU ve ue (S i) (fk * Z.of_nat (S i)) c' with
  | Some Ai, Some As => Some (Ai :: As)
  | _, _ => None
  end
end.

(** [1 + n + ... + n^(k-1)]. *)
Definition sumn (n : Z) : Z :=
  fold_right Z.add 0 (map (fun i => n ^ Z.of_nat i) (seq 0 k)).

(** (H_E) with [exp x1 <= mU 2^fU]:
    [beta^l (2^-m + mU 2^fU (n u)^k / (k! v)) + sumn n <= E]. *)
Definition window_ok (mU fU ve ue n : Z) : bool :=
  let X := mU * n ^ Z.of_nat k in
  let g := fU + lbits + ue * Z.of_nat k - ve in
  let C := 2 ^ (lbits - m) + sumn n in
  let kf := Z.of_nat (fact k) in
  if Z.leb 0 g then Z.leb (X * 2 ^ g + C * kf) (E * kf)
  else Z.leb (X + C * kf * 2 ^ (- g)) (E * kf * 2 ^ (- g)).

Definition check_line (S0 ex n : Z) (B : list Z) : bool :=
  let S1 := S0 + n in
  let ue := uexp ex in
  Z.leb 0 n && Z.leb (2 ^ 52) (Z.abs S0) && Z.ltb (Z.abs S0) (2 ^ 53) &&
  Z.leb (2 ^ 52) (Z.abs S1) && Z.ltb (Z.abs S1) (2 ^ 53) &&
  Z.ltb 0 (S0 * S1) && Nat.eqb (length B) k &&
  match encl S0 ue, encl S1 ue with
  | Some (mL0, fL0, mU0, fU0), Some (mL1, fL1, mU1, fU1) =>
    let e := Z.log2 mL0 + fL0 in
    let ve := vexp e in
    lt2 mU1 fU1 (Z.max e emin + 1) && window_ok mU1 fU1 ve ue n &&
    match coefs mL0 fL0 mU0 fU0 ve ue 0 1 k with
    | Some A =>
      forallb (fun i => Z.eqb (nth i B 0) (Pz A (Z.of_nat i) mod beta_l))
        (seq 0 k)
    | None => false
    end
  | _, _ => false
end.

```

B ExpParse.v

```
(** * Reading the lines of the table from their text

  A line of [in_lt] is kept verbatim as a primitive string:

    -0x1.62e42fefa39efp+0 6782714940072 0x15cedf0a6382cf46 ...

  that is [x0] in hexadecimal ([-]? [0x1], then [.F] with at most 13
  digits or nothing, then [p], a sign and the exponent in decimal), [n]
  in decimal, then [k * l] words in hexadecimal, each [B_i] being [l]
  words, the least significant first. [parse] turns such a line into
  [(S0, ex, n, B)] with [x0 = S0 2^(ex - 52)]; a line of another shape
  gives [None], and [check_raw] then answers [false]. *)

From Stdlib Require Import ZArith List PrimString PrimStringAxioms Uint63.
From ExpTable9 Require Import ExpCheck.
Import ListNotations.

Open Scope Z_scope.

(** ** Characters *)

Definition c_space : int := 32%uint63.    (* ' ' *)
Definition c_0 : int := 48%uint63.        (* '0' *)
Definition c_9 : int := 57%uint63.        (* '9' *)
Definition c_a : int := 97%uint63.        (* 'a' *)
Definition c_f : int := 102%uint63.       (* 'f' *)
Definition c_x : int := 120%uint63.       (* 'x' *)
Definition c_dot : int := 46%uint63.      (* '.' *)
Definition c_p : int := 112%uint63.       (* 'p' *)
Definition c_plus : int := 43%uint63.     (* '+' *)
Definition c_l : int := 49%uint63.       (* 'l' *)
Definition c_minus : int := 45%uint63.    (* '-' *)

(** The value of a hexadecimal digit, or [None]. *)
Definition hexd (c : int) : option Z :=
  if (c_0 <=? c)%uint63 && (c <=? c_9)%uint63 then Some (to_Z (c - c_0))
  else if (c_a <=? c)%uint63 && (c <=? c_f)%uint63
  then Some (to_Z (c - c_a) + 10)
  else None.

(** The value of a decimal digit, or [None]. *)
Definition decd (c : int) : option Z :=
  if (c_0 <=? c)%uint63 && (c <=? c_9)%uint63 then Some (to_Z (c - c_0))
  else None.

(** ** Numbers *)

(** The number written with the digits [cs] in base [b], most
    significant first; [None] if a character is not a digit. *)
Fixpoint digits (d : int -> option Z) (b acc : Z) (cs : list int)
  : option Z :=
  match cs with
  | [] => Some acc
  | c :: cs' =>
```

```

    match d c with
    | Some v => digits d b (acc * b + v) cs'
    | None => None
    end
end.

Definition hexv (cs : list int) : option Z :=
  match cs with [] => None | _ => digits hexd 16 0 cs end.

Definition decv (cs : list int) : option Z :=
  match cs with [] => None | _ => digits decd 10 0 cs end.

(** A word [0xhh...h]. *)
Definition wordv (cs : list int) : option Z :=
  match cs with
  | c0 :: cx :: cs' =>
    if (c0 =? c_0)%uint63 && (cx =? c_x)%uint63 then hexv cs' else None
  | _ => None
  end.

(** The characters before the first [p], and those after it. *)
Fixpoint split_p (cs : list int) : list int * list int :=
  match cs with
  | [] => ([], [])
  | c :: cs' =>
    if (c =? c_p)%uint63 then ([], cs')
    else let (a, b) := split_p cs' in (c :: a, b)
  end.

(** A signed exponent [+d...d] or [-d...d]. *)
Definition expv (cs : list int) : option Z :=
  match cs with
  | c :: cs' =>
    if (c =? c_plus)%uint63 then decv cs'
    else if (c =? c_minus)%uint63 then
      match decv cs' with Some e => Some (- e) | None => None end
    else None
  | [] => None
  end.

(** The fraction digits [F] of [0x1.F] and the exponent: [.F] may be
    absent, and [F] has at most 13 digits (trailing zeros may be left
    out); the significand is [16^13 + F 16^(13 - |F|)]. *)
Definition fracv (cs : list int) : option (Z * Z) :=
  let (fr, ex) := split_p cs in
  match fr with
  | [] => match expv ex with Some e => Some (16 ^ 13, e) | None => None end
  | cd :: fr' =>
    if (cd =? c_dot)%uint63 && Nat.leb (List.length fr') 13 then
      match digits hexd 16 1 fr', expv ex with
      | Some f, Some e => Some (f * 16 ^ Z.of_nat (13 - List.length fr'), e)
      | _, _ => None
      end
    else None
  end.
end.

```

```

(** [x0 = [-]0x1.Fp+-E]: its signed significand [S0] and its exponent,
    [x0 = S0 2^(E - 52)]. *)
Definition x0v (cs : list int) : option (Z * Z) :=
  let (sg, cs1) :=
    match cs with
    | c :: cs' => if (c =? c_minus)%uint63 then (true, cs') else (false, cs)
    | [] => (false, cs)
  end in
  match cs1 with
  | c0 :: cx :: c1 :: cs' =>
    if (c0 =? c_0)%uint63 && (cx =? c_x)%uint63 && (c1 =? c_1)%uint63 then
      match fracv cs' with
      | Some (M, e) => Some (if sg then - M else M, e)
      | None => None
    end
  else None
  | _ => None
end.

(** ** Lines *)

(** The fields of a line, separated by single spaces. *)
Fixpoint fields (cur : list int) (cs : list int) : list (list int) :=
  match cs with
  | [] => [rev cur]
  | c :: cs' =>
    if (c =? c_space)%uint63 then rev cur :: fields [] cs'
    else fields (c :: cur) cs'
  end.

(** [w_0 + w_1 beta + ... ], the words least significant first. *)
Definition wsum (ws : list Z) : Z := fold_right (fun w a => w + beta * a) 0 ws.

(** The [B_i], each of [l] words, from the list of all words. *)
Fixpoint group (c : nat) (ws : list Z) : list Z :=
  match c with
  | 0 => []
  | S c' => wsum (firstn (Z.to_nat l) ws) :: group c' (skipn (Z.to_nat l) ws)
  end.

(** All the words, or [None] if one is not a word. *)
Fixpoint wordsv (fs : list (list int)) : option (list Z) :=
  match fs with
  | [] => Some []
  | f :: fs' =>
    match wordv f, wordsv fs' with
    | Some w, Some ws => Some (w :: ws)
    | _, _ => None
    end
  end.

Definition parse (s : string) : option (Z * Z * Z * list Z) :=
  match fields [] (to_list s) with
  | fx :: fn :: fw =>
    if Nat.eqb (List.length fw) (k * Z.to_nat l) then
      match x0v fx, decv fn, wordsv fw with

```

```

    | Some (S0, ex), Some n, Some ws => Some (S0, ex, n, group k ws)
    | _, _, _ => None
  end
else None
| _ => None
end.

```

(** ** The check on the text *)

```

Definition check_raw (ls : list string) : bool :=
  Z.ltb (2 * E) beta_l && Z.leb m_lbits &&
  forallb (fun s => match parse s with
    | Some (S0, ex, n, B) => check_line S0 ex n B
    | None => false
  end) ls.

```

C ExpProof.v

(** * A [true] answer of the checker gives the six conditions *)

```

From Stdlib Require Import ZArith Reals Lia Lra List Bool.
From Flocq Require Import Core.
From ExpTable9 Require Import ExpCheck ExpParse ExpTaylor ExpArith ExpEncl.

```

Open Scope R_scope.

(** The parameters leave room for the $[2^{-m}]$ term of the window. *)

Lemma m_le_lbits : (m <= lbits)%Z.

Proof. apply Z.leb_le; vm_compute; reflexivity. Qed.

Theorem check_lineP S0 ex n B :

check_line S0 ex n B = true -> line_ok S0 ex n B.

Proof.

unfold check_line.

set (ue := uexp ex).

destruct (encl S0 ue) as [[[[mL0 fL0] mU0] fU0]] eqn:h0;

[|rewrite !andb_false_r; discriminate].

destruct (encl (S0 + n) ue) as [[[[mL1 fL1] mU1] fU1]] eqn:h1;

[|rewrite !andb_false_r; discriminate].

set (e := (Z.log2 mL0 + fL0)%Z).

destruct (coefs mL0 fL0 mU0 fU0 (vexp e) ue 0 1 k) as [A] eqn:hc;

[|rewrite !andb_false_r; discriminate].

intros h.

apply andb_prop in h as [h hm].

apply andb_prop in hm as [hm hB].

apply andb_prop in hm as [hlt hw].

apply andb_prop in h as [h hlen].

apply andb_prop in h as [h hsg].

apply andb_prop in h as [h h53'].

apply andb_prop in h as [h h52'].

apply andb_prop in h as [h h53].

apply andb_prop in h as [hn h52].

apply Z.leb_le in hn; apply Z.leb_le in h52; apply Z.ltb_lt in h53.

apply Z.leb_le in h52'; apply Z.ltb_lt in h53'; apply Z.ltb_lt in hsg.

apply Nat.eqb_eq in hlen.

```

destruct (encl_ok _ _ _ _ _ h0) as [pL0 [pU0 [lo0 up0]]].
destruct (encl_ok _ _ _ _ _ h1) as [pL1 [pU1 [lo1 up1]]].
exists e; cbv zeta; fold ue.
assert (hu : 0 < bp ue) by apply bpow_gt_0.
assert (hx1 : IZR (S0 + n) * bp ue = IZR S0 * bp ue + IZR n * bp ue)
  by (rewrite plus_IZR; ring).
repeat split.
- (* 1. u is constant *)
  exact hn.
- exact h52.
- exact h53.
- exact h52'.
- exact h53'.
- exact hsg.
- (* 2. v is constant *)
  apply Rle_trans with (IZR mL0 * bp fL0); [apply log2_le; exact pL0|].
  exact lo0.
- apply Rle_lt_trans with (IZR mU1 * bp fU1); [exact up1|].
  apply lt2_ok; [lia|exact hlt].
- (* 3. (H_A), and the table holds P(0) .. P(k-1) *)
  exists A.
  destruct (coefs_ok _ _ _ _ _ _ _ _ _ eq_refl (conj lo0 up0) hc)
    as [h1A hA].
  split; [exact h1A|]; split; [exact hlen|]; split.
  + intros i hi; destruct (hA i hi) as [hA1 hA2]; split; [exact hA1|].
    exact hA2.
  + intros i hi.
    rewrite forallb_forall in hB.
    apply Z.eqb_eq, hB, in_seq; lia.
- (* 4. (H_T) *)
  intros j hj; rewrite hx1.
  apply HT_ok; [exact hu|apply bpow_gt_0|lia].
- (* 5. (H_E) *)
  apply window_okP with (mU := mU1) (fU := fU1);
    [exact m_le_lbits|exact hn| |exact hw|.
  split; [apply Rlt_le, exp_pos|exact up1|.
Qed.

```

(** The same on the text of the lines: each one reads as a line that satisfies the six conditions. *)

```

Theorem check_rawP ls : check_raw ls = true ->
  (2 * E < beta_l)%Z /\
  forall s, In s ls -> exists S0 ex n B,
    parse s = Some (S0, ex, n, B) /\ line_ok S0 ex n B.

```

Proof.

```

unfold check_raw; intros h.
apply andb_prop in h as [h hall].
apply andb_prop in h as [h2E _].
split; [apply Z.ltb_lt; exact h2E|].
intros s hin.
rewrite forallb_forall in hall.
specialize (hall _ hin).
destruct (parse s) as [[[[S0 ex] n] B]]]; [|discriminate].
exists S0, ex, n, B; split; [reflexivity|].
apply check_lineP; exact hall.
Qed.

```

D ExpParseSpec.v

(** * What a line of the table says, and [parse] reads exactly that

A line of [in_lt] is the text

$[-]0x1[.F]pSX\ N\ 0xW_0\ 0xW_1\ \dots\ 0xW_{(k\ l - 1)}$

with single spaces between the fields (none inside [x0]): an optional minus sign, [0x1], an optional fraction [.F] with [F] 0 to 13 hexadecimal digits, [p], a sign [S] ('+' or '-') and [X] decimal digits, then [N] decimal digits and each [W_j] hexadecimal digits. With [ex = +-X] (the sign [S]), [x0 = $\pm(1 + F / 16^{|F|}) 2^{ex}$] (the optional minus sign), a missing fraction counting as [F] empty. With [S0 = $\pm(16^{13} + F 16^{(13 - |F|)})$] this is [x0 = S0 $2^{(ex - 52)}$]. The line gives [n = N], and [B_i = $w_{(i\ l)} + w_{(i\ l + 1)} \beta + \dots + w_{(i\ l + l - 1)} \beta^{(l-1)}$] for [i < k], [w_j] being the value of [W_j].

[line_text] states this; [parse_sound] proves that whatever [parse] returns is what the text says. A character is its code, an [int]. *)

From Stdlib Require Import PrimString PrimStringAxioms UInt63.

From ExpTable9 Require Import ExpCheck ExpParse.

From Stdlib Require Import ZArith List Lia.

Import ListNotations.

Open Scope Z_scope.

(** ** Digits and numbers *)

(** The characters [c_0], [c_a], ... are those of [ExpParse]. *)

(** [c] is the decimal digit of value [d]: '0' .. '9'. *)

Definition decdigit (c : int) (d : Z) : Prop :=

0 <= d <= 9 /\ to_Z c = to_Z c_0 + d.

(** [c] is the hexadecimal digit of value [d]: '0' .. '9', 'a' .. 'f'. *)

Definition hexdigit (c : int) (d : Z) : Prop :=

decdigit c d /\ (10 <= d <= 15 /\ to_Z c = to_Z c_a + (d - 10)).

(** [d_0 b^{(len-1)} + d_1 b^{(len-2)} + ... + d_{(len-1)}], most significant first. *)

Fixpoint value (b : Z) (ds : list Z) : Z :=

match ds with

| [] => 0

| d :: ds' => d * b ^ Z.of_nat (length ds') + value b ds'

end.

(** [cs] is a nonempty decimal numeral of value [v]. *)

Definition dec_text (cs : list int) (v : Z) : Prop :=

cs <> [] /\ exists ds, Forall2 decdigit cs ds /\ v = value 10 ds.

(** [cs] is a possibly empty list of hexadecimal digits of value [v]. *)

Definition hexs_text (cs : list int) (v : Z) : Prop :=

exists ds, Forall2 hexdigit cs ds /\ v = value 16 ds.

```

(** [cs] is a nonempty hexadecimal numeral of value [v]. *)
Definition hex_text (cs : list int) (v : Z) : Prop :=
  cs <> [] /\ hexs_text cs v.

(** [W] is a word [0xhh...h] of value [w]. *)
Definition word_text (W : list int) (w : Z) : Prop :=
  exists H, W = c_0 :: c_x :: H /\ hex_text H w.

(** [Xs] is a signed exponent [+X] or [-X] of value [e]. *)
Definition exp_text (Xs : list int) (e : Z) : Prop :=
  exists X v, dec_text X v /\
    (Xs = c_plus :: X /\ e = v \/ Xs = c_minus :: X /\ e = - v).

(** [f 0 + f 1 + ... + f (N-1)]. *)
Fixpoint sumZ (f : nat -> Z) (N : nat) : Z :=
  match N with 0 => 0 | S N' => sumZ f N' + f N' end.

(** ** A line *)

Definition frac_digits : nat := 13. (* hex digits of a binary64 fraction *)

(** [cs] is the line [[-]0x1[.F]pSX N W_0 ... W_(k l - 1)] of
    [(S0, ex, n, B)]. *)
Definition line_text (cs : list int) (S0 ex n : Z) (B : list Z) : Prop :=
  exists Sg Dot F f Xs N Ws ws,
  cs = Sg ++ [c_0; c_x; c_1] ++ Dot ++ F ++ c_p :: Xs ++
    c_space :: N ++ concat (map (fun W => c_space :: W) Ws) /\
    (* the fraction: absent, or a dot and 0 to 13 digits of value f *)
    (Dot = [] /\ F = [] \/ Dot = [c_dot]) /\
    hexs_text F f /\ (length F <= frac_digits)%nat /\
    (* S0 = +-(16^13 + f 16^(13 - |F|)), minus if the line starts by '-' *)
    (Sg = [] /\ S0 = 16 ^ Z.of_nat frac_digits +
      f * 16 ^ (Z.of_nat frac_digits - Z.of_nat (length F)) \/
      Sg = [c_minus] /\ S0 = - (16 ^ Z.of_nat frac_digits +
        f * 16 ^ (Z.of_nat frac_digits - Z.of_nat (length F)))) /\
    (* the exponent: ex = +X or -X *)
    exp_text Xs ex /\
    (* n = N *)
    dec_text N n /\
    (* the k l words w_j *)
    length Ws = (k * Z.to_nat l)%nat /\ Forall2 word_text Ws ws /\
    (* B_i = sum_(j < l) w_(i l + j) beta^j *)
    B = map (fun i => sumZ (fun j => nth (i * Z.to_nat l + j) ws 0 *
      beta ^ Z.of_nat j) (Z.to_nat l))
      (seq 0 k).

(** ** Proofs *)

Lemma decdP c d : decd c = Some d -> decdigit c d.
Proof.
  unfold decd, decdigit.
  destruct ((c_0 <=? c)%uint63 && (c <=? c_9)%uint63) eqn:E1; [|discriminate].
  intros [= <-]; apply andb_prop in E1 as [E1 E2].
  apply leb_spec in E1, E2.
  pose proof (to_Z_bounded c) as Hc.

```

```

change (to_Z c_0) with 48 in *; change (to_Z c_9) with 57 in *.
rewrite sub_spec; change (to_Z c_0) with 48.
rewrite Z.mod_small; lia.
Qed.

```

```

Lemma hexdP c d : hexd c = Some d -> hexdigit c d.
Proof.
unfold hexd, hexdigit.
destruct ((c_0 <=? c)%uint63 && (c <=? c_9)%uint63) eqn:E1.
{ intros H; left; apply decdP; unfold decd; rewrite E1; exact H. }
destruct ((c_a <=? c)%uint63 && (c <=? c_f)%uint63) eqn:E2; [|discriminate].
intros [= <-]; right; apply andb_prop in E2 as [E2 E3].
apply leb_spec in E2, E3.
pose proof (to_Z_bounded c) as Hc.
change (to_Z c_a) with 97 in *; change (to_Z c_f) with 102 in *.
rewrite sub_spec; change (to_Z c_a) with 97.
rewrite Z.mod_small; lia.
Qed.

```

```

Lemma digitsP d b acc cs v : digits d b acc cs = Some v ->
  exists ds, Forall2 (fun c x => d c = Some x) cs ds /\
    v = acc * b ^ Z.of_nat (length cs) + value b ds.
Proof.
revert acc; induction cs as [|c cs IH]; intros acc; cbn [digits].
- intros [= <-]; exists []; split; [constructor|simpl; ring].
- destruct (d c) as [x] eqn:Hc; [|discriminate].
  intros H; destruct (IH _ H) as [ds [H1 H2]].
  exists (x :: ds); split; [constructor; auto|].
  cbn [value List.length].
  rewrite <- (Forall2_length H1), H2, Nat2Z.inj_succ, Z.pow_succ_r by lia.
  ring.
Qed.

```

```

Lemma decvP cs v : decv cs = Some v -> dec_text cs v.
Proof.
unfold decv; destruct cs as [|c cs']; [discriminate|].
intros H; apply digitsP in H as [ds [H1 H2]].
split; [discriminate|]; exists ds; split.
- eapply Forall2_impl; [|exact H1]; intros ? ? ?; apply decdP; assumption.
- rewrite H2; ring.
Qed.

```

```

Lemma hexvP cs v : hexv cs = Some v -> hex_text cs v.
Proof.
unfold hexv; destruct cs as [|c cs']; [discriminate|].
intros H; apply digitsP in H as [ds [H1 H2]].
split; [discriminate|]; exists ds; split.
- eapply Forall2_impl; [|exact H1]; intros ? ? ?; apply hexdP; assumption.
- rewrite H2; ring.
Qed.

```

```

Lemma wordvP cs w : wordv cs = Some w -> word_text cs w.
Proof.
unfold wordv; destruct cs as [|c0 [|cx cs']]; try discriminate.
destruct (c0 =? c_0)%uint63 eqn:E0; [|discriminate].
destruct (cx =? c_x)%uint63 eqn:Ex; [|discriminate].

```

```

apply eqb_spec in E0, Ex; subst; intros H.
exists cs'; split; [reflexivity|apply hexvP; exact H].
Qed.

Lemma wordsvP fs ws : wordsv fs = Some ws -> Forall2 word_text fs ws.
Proof.
revert ws; induction fs as [|f fs IH]; intros ws; simpl.
- intros [= <-]; constructor.
- destruct (wordv f) as [w|] eqn:Hf; [|discriminate].
  destruct (wordsv fs) as [ws'|] eqn:Hfs; [|discriminate].
  intros [= <-]; constructor; [apply wordvP|apply IH]; auto.
Qed.

Lemma split_pP cs a b : split_p cs = (a, b) -> b <> [] ->
  cs = a ++ c_p :: b.
Proof.
revert a; induction cs as [|c cs IH]; intros a; simpl.
- intros [= <- <-]; contradiction.
- destruct (c =? c_p)%uint63 eqn:Ec.
  + apply eqb_spec in Ec; subst; intros [= <- <-]; reflexivity.
  + destruct (split_p cs) as [a' b'] eqn:Hs.
    intros [= <- <-] Hb; simpl; f_equal; apply IH; auto.
Qed.

Lemma expvP cs e : expv cs = Some e -> exp_text cs e.
Proof.
unfold expv; destruct cs as [|c cs']; [discriminate|].
destruct (c =? c_plus)%uint63 eqn:Ep.
- apply eqb_spec in Ep; subst; intros H.
  exists cs', e; split; [apply decvP; exact H|left; auto].
- destruct (c =? c_minus)%uint63 eqn:Em; [|discriminate].
  apply eqb_spec in Em; subst.
  destruct (decv cs') as [v|] eqn:Hv; [|discriminate].
  intros [= <-]; exists cs', v; split; [apply decvP; exact Hv|right; auto].
Qed.

(** The significand  $[16^{13} + f \cdot 16^{(13 - |F|)}]$  of  $[0x1.F]$ . *)
Definition signif (F : list int) (f : Z) : Z :=
  16 ^ Z.of_nat frac_digits +
  f * 16 ^ (Z.of_nat frac_digits - Z.of_nat (length F)).

Lemma fracvP cs M e : fracv cs = Some (M, e) ->
  exists Dot F f Xs, cs = Dot ++ F ++ c_p :: Xs /\
  (Dot = [] /\ F = [] \/ Dot = [c_dot]) /\
  hexs_text F f /\ (length F <= frac_digits)%nat /\
  M = signif F f /\ exp_text Xs e.
Proof.
unfold fracv; destruct (split_p cs) as [fr ex] eqn:Hs.
assert (Hex : ex <> [] -> cs = fr ++ c_p :: ex).
{ intros H; apply split_pP; [exact Hs|exact H]. }
destruct fr as [|cd fr'].
- destruct (expv ex) as [e'|] eqn:He; [|discriminate].
  intros [= <- <-].
  exists [], [], 0, ex; split; [|split; [|split; [|split; [|split; [|split]]]]].
  + apply Hex; intros ->; discriminate He.
  + left; auto.

```

```

+ exists []; split; [constructor|reflexivity].
+ unfold frac_digits; simpl; lia.
+ unfold signif; rewrite Z.mul_0_l, Z.add_0_r; reflexivity.
+ apply expvP; exact He.
- destruct ((cd =? c_dot)%uint63 && Nat.leb (length fr') 13) eqn:Ec;
  [|discriminate].
  apply andb_prop in Ec as [Ed Hl].
  apply eqb_spec in Ed; apply Nat.leb_le in Hl; subst cd.
  destruct (digits hexd 16 1 fr') as [f1] eqn:Hd; [|discriminate].
  destruct (expv ex) as [e'] eqn:He; [|discriminate].
  intros [= <- <-].
  apply digitsP in Hd as [ds [H1 H2]].
  exists [c_dot], fr', (value 16 ds), ex.
  split; [|split; [|split; [|split; [|split; [|split]]]]].
  + rewrite Hex; [reflexivity|intros ->; discriminate He].
  + right; reflexivity.
  + exists ds; split; [|reflexivity].
    eapply Forall2_impl; [|exact H1]; intros ? ? ?; apply hexdP; assumption.
  + exact Hl.
+ change (f1 * 16 ^ Z.of_nat (13 - length fr') =
  signif fr' (value 16 ds)).
  unfold signif; rewrite H2, (Nat2Z.inj_sub _ _ Hl); unfold frac_digits.
  set (L := Z.of_nat (length fr')).
  assert (HL : 0 <= L <= 13) by (unfold L; lia).
  replace (16 ^ Z.of_nat 13) with (16 ^ L * 16 ^ (13 - L))
    by (rewrite <- Z.pow_add_r by lia; f_equal; lia).
  change (Z.of_nat 13) with 13; ring.
+ apply expvP; exact He.
Qed.

(** [x0v] after the optional minus sign, [sg] telling if it was there. *)
Lemma x0v_body (sg : bool) (cs1 : list int) (S0 e : Z) :
  (match cs1 with
  | c0 :: cx :: c1 :: cs' =>
    if (c0 =? c_0)%uint63 && (cx =? c_x)%uint63 && (c1 =? c_1)%uint63 then
      match fracv cs' with
      | Some (M, e') => Some (if sg then - M else M, e')
      | None => None
    end
  else None
  | _ => None
  end : option (Z * Z)) = Some (S0, e) ->
  exists Dot F f Xs, cs1 = [c_0; c_x; c_1] ++ Dot ++ F ++ c_p :: Xs /\
  (Dot = [] /\ F = [] \/ Dot = [c_dot]) /\
  hexs_text F f /\ (length F <= frac_digits)%nat /\
  S0 = (if sg then - signif F f else signif F f) /\ exp_text Xs e.
Proof.
destruct cs1 as [|c0 [|cx [|c1 cs']]]; try discriminate.
destruct (c0 =? c_0)%uint63 eqn:E0; [|discriminate].
destruct (cx =? c_x)%uint63 eqn:Ex; [|discriminate].
destruct (c1 =? c_1)%uint63 eqn:E1; [|discriminate]; cbn [andb].
apply eqb_spec in E0, Ex, E1; subst.
destruct (fracv cs') as [[M e']] eqn:Hf; [|discriminate].
intros [= <- <-].
apply fracvP in Hf as (Dot & F & f & Xs & -> & HD & HF & HL & -> & HX).
exists Dot, F, f, Xs; repeat split; auto.

```

Qed.

```
Lemma x0vP cs S0 e : x0v cs = Some (S0, e) ->
  exists Sg Dot F f Xs,
  cs = Sg ++ [c_0; c_x; c_1] ++ Dot ++ F ++ c_p :: Xs /\
  (Dot = [] /\ F = [] \/ Dot = [c_dot]) /\
  hexs_text F f /\ (length F <= frac_digits)%nat /\
  (Sg = [] /\ S0 = signif F f \/ Sg = [c_minus] /\ S0 = - signif F f) /\
  exp_text Xs e.
```

Proof.

```
unfold x0v; destruct cs as [|c cs0]; [discriminate|].
destruct (c =? c_minus)%uint63 eqn:Em.
- intros H; apply (x0v_body true cs0) in H.
  destruct H as (Dot & F & f & Xs & -> & HD & HF & HL & HS & HX).
  apply eqb_spec in Em; subst.
  exists [c_minus], Dot, F, f, Xs; repeat split; auto.
- intros H; apply (x0v_body false (c :: cs0)) in H.
  destruct H as (Dot & F & f & Xs & Hc & HD & HF & HL & HS & HX).
  exists [], Dot, F, f, Xs; repeat split; auto.
```

Qed.

```
Lemma fields_nil cur cs : fields cur cs <> [].
```

Proof.

```
revert cur; induction cs as [|c cs IH]; intros cur; simpl; [discriminate|].
destruct (c =? c_space)%uint63; [discriminate|apply IH].
Qed.
```

```
Lemma fieldsP cur cs f fs : fields cur cs = f :: fs ->
```

```
  rev cur ++ cs = f ++ concat (map (fun W => c_space :: W) fs).
```

Proof.

```
revert cur f fs; induction cs as [|c cs IH]; intros cur f fs; simpl.
- intros [= <- <-]; simpl; rewrite !app_nil_r; reflexivity.
- destruct (c =? c_space)%uint63 eqn:Ec.
  + apply eqb_spec in Ec; subst; intros [= <- <-].
    destruct (fields [] cs) as [|g gs] eqn:Hg; [now apply fields_nil in Hg|].
    apply IH in Hg; simpl in Hg; rewrite Hg; reflexivity.
  + intros H; apply IH in H; rewrite <- H; simpl.
    rewrite <- app_assoc; reflexivity.
```

Qed.

```
Lemma sumZ_ext f g N : (forall j, f j = g j) -> sumZ f N = sumZ g N.
```

Proof. intros H; induction N; simpl; [reflexivity|rewrite IHN, H; ring]. Qed.

```
Lemma sumZ_shift f N : sumZ f (S N) = f 0 + sumZ (fun j => f (S j)) N.
```

Proof.

```
induction N; [simpl; ring|].
change (sumZ f (S (S N))) with (sumZ f (S N) + f (S N)).
rewrite IHN; simpl; ring.
Qed.
```

```
Lemma sumZ_scale c f N :
```

```
  sumZ (fun j => c * f j) N = c * sumZ f N.
```

Proof. induction N; simpl; [ring|rewrite IHN; ring]. Qed.

```
Lemma wsum_firstn n ws : wsum (firstn n ws) =
```

```
  sumZ (fun j => nth j ws 0 * beta ^ Z.of_nat j) n.
```

```

Proof.
revert ws; induction n as [|n IH]; intros ws; [reflexivity|].
rewrite sumZ_shift; destruct ws as [|w ws].
- rewrite (sumZ_ext _ (fun j => 0 * beta ^ Z.of_nat (S j)))
  by (intros [|j]; reflexivity).
  rewrite sumZ_scale; unfold wsum; cbn [firstn nth fold_right]; ring.
- change (wsum (firstn (S n) (w :: ws))) with
  (w + beta * wsum (firstn n ws)).
  rewrite IH.
  rewrite (sumZ_ext (fun j => nth (S j) (w :: ws) 0 * beta ^ Z.of_nat (S j))
    (fun j => beta * (nth j ws 0 * beta ^ Z.of_nat j))).
  + rewrite (sumZ_scale beta (fun j => nth j ws 0 * beta ^ Z.of_nat j)).
    cbn [nth]; change (Z.of_nat 0) with 0; ring.
  + intros j; cbn [nth]; rewrite Nat2Z.inj_succ, Z.pow_succ_r by lia; ring.
Qed.

```

```

Lemma groupP c ws : group c ws =
  map (fun i => sumZ (fun j => nth (i * Z.to_nat l + j) ws 0 *
    beta ^ Z.of_nat j) (Z.to_nat l))
    (seq 0 c).

```

```

Proof.
revert ws; induction c as [|c IH]; intros ws; [reflexivity|].
change (group (S c) ws) with
  (wsum (firstn (Z.to_nat l) ws) :: group c (skipn (Z.to_nat l) ws)).
rewrite IH, wsum_firstn; cbn [seq map]; rewrite <- seq_shift, map_map.
f_equal; apply map_ext; intros i; apply sumZ_ext; intros j.
rewrite nth_skipn; f_equal; f_equal; lia.
Qed.

```

(** ** The theorem *)

```

(** Whatever [parse] returns is what the line says. *)
Theorem parse_sound s S0 ex n B :
  parse s = Some (S0, ex, n, B) -> line_text (to_list s) S0 ex n B.

```

```

Proof.
unfold parse.
destruct (fields [] (to_list s)) as [|fx [|fn fw]] eqn:Hf; try discriminate.
destruct (Nat.eqb_spec (length fw) (k * Z.to_nat l)) as [Hl|];
  [|discriminate|.
destruct (x0v fx) as [[S e]] eqn:Hx; [|discriminate|.
destruct (decv fn) as [n'|] eqn:Hn; [|discriminate|.
destruct (wordsv fw) as [ws|] eqn:Hw; [|discriminate|.
intros [= <- <- <- <-].
apply fieldsP in Hf; simpl in Hf.
apply x0vP in Hx as (Sg & Dot & F & f & Xs & -> & HD & HF & HL & HS & HX).
exists Sg, Dot, F, f, Xs, fn, fw, ws.
split; [|split; [|split; [|split; [|split; [|split; [|split; [|split; [|split;
  [|split]]]]]]]]]; auto.
- rewrite Hf, <- !app_assoc; reflexivity.
- apply decvP; exact Hn.
- apply wordsvP; exact Hw.
- exact (groupP k ws).
Qed.

```

(** ** A short example *)

```

(** Six words [0x0], an [l]-word group of zeros. *)
Definition zeros6 : string := " 0x0 0x0 0x0 0x0 0x0 0x0"%pstring.

(** Eight groups of zeros. *)
Definition zeros48 : string :=
  cat zeros6 (cat zeros6 (cat zeros6 (cat zeros6
    (cat zeros6 (cat zeros6 (cat zeros6 zeros6)))))).

(** [x0 = -0x1.8p-3 = -3 2^51 2^(-3 - 52)], [n = 3], words [1, 2, 0, ...]. *)
Definition example_line : string :=
  cat "-0x1.8p-3 3 0x1 0x2 0x0 0x0 0x0 0x0"%pstring zeros48.

Example parse_example : parse example_line =
  Some (- 3 * 2 ^ 51, -3, 3, [1 + 2 * beta; 0; 0; 0; 0; 0; 0; 0; 0]).
Proof. vm_compute; reflexivity. Qed.

(** [x0 = 0x1p+0 = 1] without a fraction: [S0 = 16^13], [ex = 0]. *)
Example parse_example2 :
  parse (cat "0x1p+0 7 0x0 0x0 0x0 0x0 0x0 0x0"%pstring zeros48) =
  Some (16 ^ 13, 0, 7, [0; 0; 0; 0; 0; 0; 0; 0; 0]).
Proof. vm_compute; reflexivity. Qed.

```

E ExpArith.v

```

(** * The integer tests of the checker, read in the reals *)

From Stdlib Require Import ZArith Reals Lia Lra List.
From Flocq Require Import Core.
From ExpTable9 Require Import ExpCheck.

Open Scope R_scope.

(** [flr] is the floor it is named after. *)
Lemma flrE p g fk h : (0 < fk)%Z -> (0 <= h)%Z ->
  flr p g fk h = Zfloor (IZR p * bp g / IZR fk + IZR h / 2).
Proof.
  intros Hfk Hh; unfold flr, bp.
  assert (Hfk' : IZR fk <> 0) by (apply not_0_IZR; lia).
  destruct (Z.leb_spec 0 g) as [Hg | Hg].
  - rewrite <- Zfloor_div by lia; f_equal.
    rewrite <- (IZR_Zpower radix2 g) by lia; simpl radix_val.
    rewrite !mult_IZR, plus_IZR, !mult_IZR; field; auto.
  - rewrite <- Zfloor_div.
    2: { apply Z.neq_mul_0; split; [lia]; apply Z.pow_nonzero; lia. }
    f_equal.
    (* a negative power of two is the inverse of an integer *)
    assert (Hq : bpow radix2 g = / IZR (2 ^ (- g))).
    { rewrite (IZR_Zpower radix2) by lia; rewrite bpow_opp, Rinv_inv; easy. }
    assert (H2 : IZR (2 ^ (- g)) <> 0).
    { apply not_0_IZR; apply Z.pow_nonzero; lia. }
    rewrite Hq, !mult_IZR, plus_IZR, !mult_IZR; field; auto.
Qed.

Lemma lt2_ok p f g : (1 <= p)%Z -> lt2 p f g = true -> IZR p * bp f < bp g.
Proof.

```

```

intros Hp; unfold lt2, bp.
destruct (Z.leb_spec g f) as [Hgf | Hgf]; [discriminate|].
intros Hlt; apply Z.ltb_lt in Hlt.
replace (bpow radix2 g) with (bpow radix2 ((g - f) + f)) by (f_equal; ring).
rewrite bpow_plus; apply Rmult_lt_compat_r; [apply bpow_gt_0|].
rewrite <- (IZR_Zpower radix2) by lia; apply IZR_lt; exact Hlt.
Qed.

```

Lemma log2_le p f : (0 < p)%Z -> bp (Z.log2 p + f) <= IZR p * bp f.
Proof.

```

intros Hp; unfold bp; rewrite bpow_plus.
apply Rmult_le_compat_r; [apply Rlt_le, bpow_gt_0|].
rewrite <- (IZR_Zpower radix2) by (apply Z.log2_nonneg).
apply IZR_le; destruct (Z.log2_spec p Hp) as [H _]; exact H.
Qed.

```

(** [beta^l] is the power of two of exponent [lbits]. *)
Lemma beta_lE : beta_l = (2 ^ lbits)%Z.
Proof. vm_compute; reflexivity. Qed.

Lemma lbits_ge0 : (0 <= lbits)%Z.
Proof. vm_compute; discriminate. Qed.

Lemma beta_lR : IZR beta_l = bp lbits.
Proof.
rewrite beta_lE; unfold bp; rewrite (IZR_Zpower radix2); [easy|].
exact lbits_ge0.
Qed.

(** A power of a power of two. *)
Lemma bp_pow a i : bp a ^ i = bp (a * Z.of_nat i).
Proof.
induction i as [|i IH]; [now rewrite Z.mul_0_r|].
rewrite Nat2Z.inj_succ, <- Z.add_1_r, Z.mul_add_distr_l, Z.mul_1_r.
unfold bp in *; rewrite bpow_plus, <- IH; simpl; ring.
Qed.

(** Moving the start of an integer sum. *)
Lemma fold_addE (s : list Z) a :
 fold_right Z.add a s = (fold_right Z.add 0 s + a)%Z.
Proof. induction s as [|x s IH]; simpl; lia. Qed.

(** [sumR] of powers of an integer, in integers. *)
Lemma sumR_powE n N : sumR (fun i => IZR n ^ i) N =
 IZR (fold_right Z.add 0 (map (fun i => n ^ Z.of_nat i) (seq 0 N)))%Z.
Proof.
induction N as [|N IH]; [easy|].
rewrite seq_S, map_app, fold_right_app; simpl.
rewrite fold_addE, plus_IZR, <- IH, Z.add_0_r, pow_IZR; easy.
Qed.

(** A power of two, as an integer. *)
Lemma bp_IZR g : (0 <= g)%Z -> bp g = IZR (2 ^ g).
Proof.
intros Hg; unfold bp; rewrite <- (IZR_Zpower radix2) by exact Hg; easy.
Qed.

```

(** A negative power of two, as the inverse of an integer. *)
Lemma bp_IZRN g : (g < 0)%Z -> bp g = / IZR (2 ^ (- g)).
Proof.
intros Hg; unfold bp; rewrite (IZR_Zpower radix2) by lia.
rewrite bpow_opp, Rinv_inv; easy.
Qed.

(** [window_ok] in the reals, before the bound on [exp x1]. *)
Lemma window_okR mU fU ve ue n : window_ok mU fU ve ue n = true ->
  IZR (2 ^ (lbits - m) + sumn n) + IZR (mU * n ^ Z.of_nat k) *
  bp (fU + lbits + ue * Z.of_nat k - ve) / IZR (Z.of_nat (fact k))
  <= IZR E.
Proof.
unfold window_ok; cbv zeta.
set (X := (mU * n ^ Z.of_nat k)%Z).
set (g := (fU + lbits + ue * Z.of_nat k - ve)%Z).
set (C := (2 ^ (lbits - m) + sumn n)%Z).
set (kf := Z.of_nat (fact k)).
assert (Hkf : (0 < kf)%Z) by (pose proof (lt_0_fact k); lia).
assert (HkfR : 0 < IZR kf) by (apply IZR_lt; exact Hkf).
clearbody X g C kf; generalize E; intros EE.
destruct (Z.leb_spec 0 g) as [Hg | Hg]; intros H; apply Z.leb_le, IZR_le in H;
  rewrite ?plus_IZR, ?mult_IZR, ?plus_IZR, ?mult_IZR in H.
- rewrite bp_IZR by exact Hg.
  (* field and lra only see real variables, or the kernel check is slow *)
  generalize (IZR X) (IZR C) (IZR (2 ^ g)) (IZR EE) H.
  generalize (IZR kf) HkfR; clear; intros kr Hk xr cr pr er H.
  apply Rmult_le_reg_r with (1 := Hk).
  replace ((cr + xr * pr / kr) * kr) with (xr * pr + cr * kr) by (field; lra).
  exact H.
- rewrite bp_IZRN by exact Hg.
  assert (HP : 0 < IZR (2 ^ (- g))).
  { apply IZR_lt; apply Z.pow_pos_nonneg; lia. }
  generalize (IZR X) (IZR C) (IZR (2 ^ (- g))) (IZR EE) H HP.
  generalize (IZR kf) HkfR; clear; intros kr Hk xr cr pr er H Hp.
  apply Rmult_le_reg_r with (r := kr * pr); [now apply Rmult_lt_0_compat|].
  replace ((cr + xr * / pr / kr) * (kr * pr))
    with (xr + cr * kr * pr) by (field; lra).
  lra.
Qed.

(** (H_E), from an upper bound [mU 2^fU] of [y = exp x1]. *)
Lemma window_okP mU fU ve ue n y :
  (m <= lbits)%Z -> (0 <= n)%Z -> 0 <= y <= IZR mU * bp fU ->
  window_ok mU fU ve ue n = true ->
  IZR beta_l * (bp (- m) + y * (IZR n * bp ue) ^ k / (INR (fact k) * bp ve))
  + sumR (fun i => IZR n ^ i) k <= IZR E.
Proof.
intros Hm Hn Hy Hw.
eapply Rle_trans; [| exact (window_okR _ _ _ _ Hw)].
rewrite sumR_powE; fold (sumn n).
rewrite plus_IZR, mult_IZR, <- pow_IZR, <- INR_IZR_INZ.
rewrite <- bp_IZR by lia.
rewrite beta_lR.
rewrite Rpow_mult_distr, bp_pow.

```

```

replace (bp (lbits - m)) with (bp lbits * bp (- m))
  by (unfold bp; rewrite <- bpow_plus; f_equal; ring).
replace (bp (fU + lbits + ue * Z.of_nat k - ve))
  with (bp fU * bp lbits * bp (ue * Z.of_nat k) / bp ve)
  by (unfold bp, Rdiv; rewrite <- bpow_opp, <- !bpow_plus; f_equal; ring).
assert (HB : 0 < bp lbits) by apply bpow_gt_0.
assert (HU : 0 < bp (ue * Z.of_nat k)) by apply bpow_gt_0.
assert (HV : 0 < bp ve) by apply bpow_gt_0.
assert (HK : 0 < INR (fact k)) by apply INR_fact_lt_0.
assert (HN : 0 <= IZR n ^ k) by (apply pow_le, IZR_le; exact Hn).
(* only real variables are left *)
revert Hy HB HU HV HK HN.
generalize (bp lbits) (bp (- m)) (bp (ue * Z.of_nat k)) (bp ve).
generalize (INR (fact k)) (IZR n ^ k) (IZR (sumn n)) (IZR mU) (bp fU).
clear; intros K N S F1 F2 B M U V Hy HB HU HV HK HN.
assert (Hc : 0 <= N * U * B / (K * V)).
{ unfold Rdiv; apply Rmult_le_pos.
  - apply Rmult_le_pos; [apply Rmult_le_pos]; lra.
  - apply Rlt_le, Rinv_0_lt_compat, Rmult_lt_0_compat; easy. }
replace (B * (M + y * (N * U) / (K * V)) + S)
  with (B * M + S + y * (N * U * B / (K * V))) by (field; lra).
replace (B * M + S + F1 * N * (F2 * B * U / V) / K)
  with (B * M + S + (F1 * F2) * (N * U * B / (K * V))) by (field; lra).
apply Rplus_le_compat_l, Rmult_le_compat_r; [exact Hc | apply Hy].
Qed.

(** A floor that is the same at both ends is the same in between. *)
Lemma Zfloor_mid a b d : a <= b <= d -> Zfloor a = Zfloor d ->
  Zfloor b = Zfloor a.
Proof.
intros [Hab Hbd] Had.
apply Zfloor_le in Hab; apply Zfloor_le in Hbd; lia.
Qed.

(** (H_A) for one coefficient, from an enclosure of [y = exp x0]. *)
Lemma coefA_ok mL fL mU fU ve ue i fk Ai y :
  fk = Z.of_nat (fact i) ->
  IZR mL * bp fL <= y <= IZR mU * bp fU ->
  coefA mL fL mU fU ve ue i fk = Some Ai ->
  (0 <= Ai < beta_l)%Z /\
  Rabs (IZR Ai - IZR beta_l *
    fracR (y * bp ue ^ i / INR (fact i) / bp ve)) < 1.
Proof.
intros Hfk Hy H.
assert (Hfk0 : (0 < fk)%Z) by (pose proof (lt_0_fact i); lia).
unfold coefA in H; cbv zeta in H.
set (gi := (lbits + ue * Z.of_nat i - ve)%Z) in H.
set (r := flr mL (fL + gi) fk 1) in H.
set (q := flr mL (fL + gi - lbits) fk 0) in H.
destruct ( _ && _ ) eqn:Hb; [|discriminate].
injection H as <-.
rewrite !Bool.andb_true_iff, Z.eqb_eq, Z.eqb_eq, Z.leb_le, Z.ltb_lt in Hb.
destruct Hb as [[H1 H2] H3] H4].
split; [lia].
unfold r, q in *.
rewrite !flrE in H1, H2 by lia.

```

```

rewrite !flrE by lia.
clear H3 H4.
assert (HfkR : 0 < IZR fk) by (apply IZR_lt; exact Hfk0).
assert (HB : 0 < bp lbits) by apply bpow_gt_0.
(* the argument of fracR is  $y (2^{gi} / i!) / \beta^l$  *)
assert (Hc :  $y * bp \text{ ue } ^i / \text{INR (fact i) / bp ve} =$ 
       $y * (bp \text{ gi} / \text{IZR fk}) / bp \text{ lbits}$ ).
{ rewrite INR_IZR_INZ, <- Hfk; rewrite bp_pow.
  unfold gi; replace (lbits + ue * Z.of_nat i - ve)%Z
    with (lbits + ue * Z.of_nat i + - ve)%Z by ring.
  assert (HV : 0 < bp ve) by apply bpow_gt_0.
  unfold bp in *; rewrite !bpow_plus, bpow_opp; field; lra. }
rewrite Hc; clear Hc; clearbody gi.
assert (Ha : forall z f, IZR z * bp (f + gi) / IZR fk =
      IZR z * bp f * (bp gi / IZR fk)).
{ intros z f; unfold bp; rewrite bpow_plus; field; lra. }
assert (Hb : forall z f, IZR z * bp (f + gi - lbits) / IZR fk + 0 / 2 =
      IZR z * bp f * (bp gi / IZR fk) / bp lbits).
{ intros z f; replace (f + gi - lbits)%Z with (f + gi + - lbits)%Z by ring.
  unfold bp; rewrite !bpow_plus, bpow_opp.
  unfold bp in HB; field; lra. }
rewrite !Ha, !Hb in *; clear Ha Hb.
assert (Hc : 0 < bp gi / IZR fk).
{ apply Rdiv_lt_0_compat; [apply bpow_gt_0 | exact HfkR]. }
rewrite minus_IZR, mult_IZR, beta_lR.
unfold fracR.
(* only real variables are left *)
revert Hy H1 H2 HB Hc.
generalize (IZR mL * bp fL) (IZR mU * bp fU) (bp gi / IZR fk) (bp lbits).
clear; intros lo hi c B Hy H1 H2 HB Hc.
assert (Hlo : lo * c <= y * c) by (apply Rmult_le_compat_r; lra).
assert (Hhi : y * c <= hi * c) by (apply Rmult_le_compat_r; lra).
rewrite <- (Zfloor_mid (lo * c / B) (y * c / B) (hi * c / B));
  [| | exact H2].
2: { unfold Rdiv; split; apply Rmult_le_compat_r; try lra;
    apply Rlt_le, Rinv_0_lt_compat; exact HB. }
rewrite <- (Zfloor_mid (lo * c + 1 / 2) (y * c + 1 / 2) (hi * c + 1 / 2));
  [| lra | exact H1].
pose proof (Zfloor_lb (y * c + 1 / 2)).
pose proof (Zfloor_ub (y * c + 1 / 2)).
replace (B * (y * c / B - IZR (Zfloor (y * c / B))))
  with (y * c - IZR (Zfloor (y * c / B)) * B) by (field; lra).
apply Rabs_def1; lra.
Qed.

(** (H_A) for the [c] coefficients from the [i]-th on. *)
Lemma coefs_ok mL fL mU fU ve ue y c i fk As :
  fk = Z.of_nat (fact i) ->
  IZR mL * bp fL <= y <= IZR mU * bp fU ->
  coefs mL fL mU fU ve ue i fk c = Some As ->
  length As = c /\
  forall j, (j < c)%nat ->
    (0 <= nth j As 0%Z < beta_l)%Z /\
    Rabs (IZR (nth j As 0%Z) - IZR beta_l *
      fracR (y * bp ue ^ (i + j) / INR (fact (i + j)) / bp ve)) < 1.
Proof.

```

```

intros Hfk Hy; revert i fk As Hfk.
induction c as [|c IH]; intros i fk As Hfk H; cbn [coefs] in H.
- injection H as <-; split; [easy | intros j Hj; lia].
- destruct (coefA mL fL mU fU ve ue i fk) as [Ai] eqn:HA; [|discriminate].
  destruct (coefs mL fL mU fU ve ue (S i) (fk * Z.of_nat (S i)) c) as [As']
    eqn:HAs; [|discriminate].
  injection H as <-.
  (* the next factorial *)
  assert (Hfk' : (fk * Z.of_nat (S i))%Z = Z.of_nat (fact (S i))).
  { change (fact (S i)) with (S i * fact i)%nat.
    rewrite Nat2Z.inj_mul, Hfk; ring. }
  destruct (IH _ _ _ Hfk' HAs) as [Hl HAs'].
  split; [simpl; lia].
  intros [|j] Hj.
  + rewrite Nat.add_0_r; exact (coefA_ok _ _ _ _ _ _ _ _ _ Hfk Hy HA).
  + replace (i + S j)%nat with (S i + j)%nat by lia.
    apply HAs'; lia.
Qed.

```

F ExpEnc1.v

```

(** * The enclosure of exp given by Interval *)

From Stdlib Require Import ZArith Reals Lia Lra.
From Flocq Require Import Core.
From Interval Require Import Xreal Basic Specific_bigint Specific_ops.
From ExpTable9 Require Import ExpCheck.

Open Scope R_scope.

(* A float with a radix-2 exponent is its signed significand times 2^e. *)
Lemma FtoR_sgn s p e :
  FtoR radix2 s p e = IZR (if s then Z.neg p else Z.pos p) * bp e.
Proof.
unfold bp; destruct s, e as [|q|q]; cbv beta iota zeta delta [FtoR bpow].
all: try rewrite mult_IZR; try lra; reflexivity.
Qed.

(* A positive float with a radix-2 exponent is p 2^e. *)
Lemma FtoR_pos p e : FtoR radix2 false p e = IZR (Z.pos p) * bp e.
Proof. exact (FtoR_sgn false p e). Qed.

(* The value of a float read by getS. *)
Lemma getS_val f p e : getS f = Some (p, e) ->
  SFBI2.toX f = Xreal (IZR p * bp e).
Proof.
unfold getS, SFBI2.toX.
destruct (SFBI2.toF f) as [| |s p' e']; try discriminate.
intros H; injection H; intros <- <-; simpl FtoX; rewrite <- FtoR_sgn.
reflexivity.
Qed.

(* The value of a float read by getF. *)
Lemma getF_val f p e : getF f = Some (p, e) ->
  SFBI2.toX f = Xreal (IZR (Z.pos p) * bp e).

```

```

Proof.
unfold getF, SFBI2.toX.
destruct (SFBI2.toF f) as [| [|] p' e']; try discriminate.
intros H; injection H; intros <- <-; simpl FtoX; rewrite <- FtoR_pos.
reflexivity.
Qed.

Lemma encl_ok N ue mL fL mU fU : encl N ue = Some (mL, fL, mU, fU) ->
  (0 < mL)%Z /\ (0 < mU)%Z /\
  IZR mL * bp fL <= exp (IZR N * bp ue) <= IZR mU * bp fU.
Proof.
unfold encl.
destruct (SFBI2.valid_lb (xpt N ue)) eqn:VL; [|discriminate|.
destruct (SFBI2.valid_ub (xpt N ue)) eqn:VU; [|discriminate|.
destruct (getS (xpt N ue)) as [|p e|] eqn:GX; [|discriminate|.
destruct (Z.eqb_spec p N) as [HN|]; [|discriminate|.
destruct (Z.eqb_spec e ue) as [He|]; [|discriminate|.
simpl andb; cbv iota.
(* The point interval contains N 2^ue. *)
assert (Hc : Interval.contains
  (I.convert (Float.Ibnd (xpt N ue) (xpt N ue))) (Xreal (IZR N * bp ue))).
{ unfold I.convert; change I.F.valid_lb with SFBI2.valid_lb.
  change I.F.valid_ub with SFBI2.valid_ub; rewrite VL, VU.
  change I.F.toX with SFBI2.toX; rewrite (getS_val _ _ _ GX), HN, He.
  simpl; lra. }
generalize (I.exp_correct iprec _ _ Hc).
destruct (I.exp iprec (Float.Ibnd (xpt N ue) (xpt N ue))) as [|lo up|];
  [discriminate|].
unfold I.convert.
change I.F.valid_lb with SFBI2.valid_lb.
change I.F.valid_ub with SFBI2.valid_ub.
change (SFBI2.valid_lb lo) with true; rewrite Bool.andb_true_l.
destruct (SFBI2.valid_ub up); [|intros _ H; discriminate H|.
destruct (getF lo) as [|pL fL'|] eqn:GL; [|intros _ H; discriminate H|.
destruct (getF up) as [|pU fU'|] eqn:GU; [|intros _ H; discriminate H|.
change I.F.toX with SFBI2.toX.
rewrite (getF_val _ _ _ GL), (getF_val _ _ _ GU).
intros [H1 H2] H; injection H; intros <- <- <- <-.
split; [lia|split; [lia|]].
split; assumption.
Qed.

```

G ExpTaylor.v

```

(** * The Taylor remainder of exp: condition 4, (H_T) *)

From Stdlib Require Import ZArith Reals Lia Lra.
From Coquelicot Require Import Coquelicot.
From ExpTable9 Require Import ExpCheck.

Open Scope R_scope.

(* Every derivative of exp is exp. *)
Lemma Derive_n_exp (m : nat) (t : R) : Derive_n exp m t = exp t.
Proof.

```

```

revert t; induction m as [|m IH]; intros t; [reflexivity|].
simpl; rewrite (Derive_ext _ _ IH).
now apply is_derive_unique, is_derive_exp.
Qed.

(* exp is differentiable [m] times everywhere. *)
Lemma ex_derive_n_exp (m : nat) (t : R) : ex_derive_n exp m t.
Proof.
destruct m as [|m]; [exact I|simpl].
apply (ex_derive_ext exp); [intros s; now rewrite Derive_n_exp|].
eexists; apply is_derive_exp.
Qed.

(* [sumR] with [N + 1] terms is the Stdlib [sum_f_R0] up to [N]. *)
Lemma sumR_sum_f (f : nat -> R) (N : nat) : sumR f (S N) = sum_f_R0 f N.
Proof.
induction N as [|N IH]; [simpl; ring|].
change (sumR f (S N) + f (S N) = sum_f_R0 f N + f (S N)); now rewrite IH.
Qed.

(** For [h >= 0], the Taylor polynomial of degree [N-1] of [exp] at [x0]
    is below [exp (x0 + h)] by at most [exp (x0 + h) h^N / N!]. *)
Lemma taylor_exp (x0 h : R) (N : nat) : 0 <= h ->
  Rabs (exp (x0 + h) - sumR (fun i => exp x0 * h ^ i / INR (fact i)) N)
  <= exp (x0 + h) * h ^ N / INR (fact N).
Proof.
intros Hh.
destruct N as [|N].
  simpl; rewrite Rminus_0_r, Rabs_right by (left; apply exp_pos); lra.
destruct Hh as [Hh|<-].
2:{ (* At [h = 0] the sum is [exp x0] and the bound is [0]. *)
  rewrite Rplus_0_r, sumR_sum_f, pow_i by lia.
  replace (sum_f_R0 _ N) with (exp x0).
  rewrite Rminus_diag, Rabs_R0, Rmult_0_r; unfold Rdiv; lra.
  induction N as [|N IH]; [simpl; field|].
  rewrite tech5, <- IH, pow_i by lia; unfold Rdiv; ring. }
(* Lagrange: the remainder is [h^(N+1) / (N+1)! exp z], [z] in the range. *)
destruct (Taylor_Lagrange exp N x0 (x0 + h)) as [z [Hz Hz']]; [lra|].
  intros t _ m _; apply ex_derive_n_exp.
rewrite sumR_sum_f.
replace (sum_f_R0 _ N) with
  (sum_f_R0 (fun m => (x0 + h - x0) ^ m / INR (fact m) * Derive_n exp m x0)
    N).
2:{ apply sum_eq; intros i _; rewrite Derive_n_exp.
  replace (x0 + h - x0) with h by ring; unfold Rdiv; ring. }
rewrite Hz' at 1; rewrite Derive_n_exp.
replace (x0 + h - x0) with h by ring.
assert (0 < INR (fact (S N))) by apply INR_fact_lt_0.
assert (0 < h ^ S N) by (apply pow_lt; lra).
rewrite Rplus_minus_l, Rabs_right.
2:{ apply Rle_ge; unfold Rdiv; apply Rmult_le_pos; [|left; apply exp_pos|.
  apply Rmult_le_pos; [lra|left; apply Rinv_0_lt_compat; lra]. }
rewrite (Rmult_comm (exp _)); unfold Rdiv.
assert (exp z < exp (x0 + h)) by (apply exp_increasing; lra).
assert (0 < / INR (fact (S N))) by (apply Rinv_0_lt_compat; lra).
assert (0 < h ^ S N * / INR (fact (S N))) by (apply Rmult_lt_0_compat; lra).

```

```

nra.
Qed.

(* Two sums with equal terms are equal. *)
Lemma sumR_ext (f g : nat -> R) (N : nat) :
  (forall i, f i = g i) -> sumR f N = sumR g N.
Proof.
intros Hfg; induction N as [|N IH]; simpl; [easy|now rewrite IH, Hfg].
Qed.

(* A common divisor comes out of the sum. *)
Lemma sumR_div (f : nat -> R) (v : R) (N : nat) :
  sumR (fun i => f i / v) N = sumR f N / v.
Proof.
induction N as [|N IH]; simpl; [|rewrite IH]; unfold Rdiv; ring.
Qed.

(** Condition 4 of [line_ok], for [j] in [[0, n]]. *)
Lemma HT_ok (x0 u v : R) (n j : Z) : 0 < u -> 0 < v -> (0 <= j <= n)%Z ->
  Rabs (exp (x0 + IZR j * u) / v - sumR (fun i => a x0 u i / v * IZR j ^ i) k)
  <= exp (x0 + IZR n * u) * (IZR n * u) ^ k / (INR (fact k) * v).
Proof.
intros Hu Hv [Hj Hjn].
apply IZR_le in Hj; apply IZR_le in Hjn.
assert (Hh : 0 <= IZR j * u) by (apply Rmult_le_pos; lra).
(* The sum is the Taylor sum at [h = j u], divided by [v]. *)
rewrite (sumR_ext _ (fun i => exp x0 * (IZR j * u) ^ i / INR (fact i) / v)).
2:{ intros i; unfold a; rewrite Rpow_mult_distr; field.
  split; [apply not_0_INR, fact_neq_0|lra]. }
rewrite (sumR_div (fun i => exp x0 * (IZR j * u) ^ i / INR (fact i))).
set (Sj := sumR _ k).
replace (exp (x0 + IZR j * u) / v - Sj / v) with
  ((exp (x0 + IZR j * u) - Sj) / v) by (field; lra).
rewrite Rabs_div, (Rabs_right v) by lra.
assert (0 < INR (fact k)) by apply INR_fact_lt_0.
replace (_ / (INR (fact k) * v)) with
  (exp (x0 + IZR n * u) * (IZR n * u) ^ k / INR (fact k) / v)
  by (field; lra).
apply Rmult_le_compat_r; [left; apply Rinv_0_lt_compat; lra|].
eapply Rle_trans; [apply taylor_exp; lra|].
(* The bound grows with [j], so [j = n] is the worst case. *)
unfold Rdiv; apply Rmult_le_compat_r; [left; apply Rinv_0_lt_compat; lra|].
apply Rmult_le_compat.
- left; apply exp_pos.
- now apply pow_le.
- destruct (Rle_lt_or_eq_dec _ _ Hjn) as [Hlt|Heq]; [|rewrite Heq; lra].
  left; apply exp_increasing; nra.
- apply pow_incr; split; [lra|]; nra.
Qed.

```